Cloud Computing

Foundations and Distributed Systems

Lecture 01 - Part 1

Academic Year 2025/2026

DATA SCIENCE AND ARTIFICIAL INTELLIGENCE – University of Trieste

Learning Objectives

By the end of this lecture, you will be able to:

- Trace the historical evolution from mainframes to cloud
- Explain distributed systems principles underlying cloud
- Apply the CAP theorem to reason about trade-offs
- Distinguish between consistency models
- Articulate why cloud emerged as a paradigm synthesis

Why Study Cloud Foundations?

"Don't be a user of pre-cooked tools that you consider as black-boxes"

Without understanding foundations:

- Cannot predict system behavior under failure
- Cannot debug distributed problems
- Cannot make informed architectural trade-offs

The Central Question

Why does cloud computing exist?

The answer is NOT simply "computing over the internet"

Cloud solves specific problems prior paradigms could not:
Elastic resource allocation • Economic efficiency • Operational complexity

Evolution of Computing Paradigms

A 70-year journey to cloud computing

Mainframes	Personal	Client-Server	Cluster/Grid	Cloud
1950s-70s	1980s-90s	1990s	2000s	2006+

Cloud computing is an evolution, not a revolution

Mainframe Era (1950s-1970s)

Characteristics

- Centralized computing
- Time-sharing among multiple users
- Access via "dumb" terminals
- Expensive hardware → maximize utilization
- Batch processing and job queues

Mainframe Lessons for Cloud

The parallels are striking

Time-sharing	→	**Multi-tenancy**	Multiple users/customers share resources
Centralized resources	→	**Resource pooling**	Efficient use of expensive infrastructure
Terminals	→	**Web browsers**	Thin clients access remote computation
Batch processing	→	**Serverless / Batch**	Submit jobs, get results later

History doesn't repeat, but it rhymes

Personal Computing (1980s-1990s)

The Decentralization Revolution

What Changed

- Dedicated computing per user
- Local software installation
- Data on local hard drives

Problems Introduced

- "It works on my machine"
- < 5% average CPU utilization
- Maintenance burden on users

Democratized access, but created inefficiency and inconsistency

Cluster Computing (1990s-2000s)

Scaling beyond single machines

Key Innovation: Horizontal Scaling

- Multiple commodity machines working together
- High-speed local network (LAN) interconnect
- Homogeneous hardware for simplicity
- Parallel processing frameworks (MPI, later MapReduce)

Cloud inherits: Scale out (add machines) rather than scale up (bigger machine)

What is a Cluster?

Cluster classification

High availability clusters (HA) (Linux)

- Mission critical applications
- High-availability clusters (also known as Failover Clusters) are implemented for the purpose of improving the availability of services which the cluster provides.
- provide redundancy
- eliminate single points of failure.

Network Load balancing clusters

- operate by distributing a workload evenly over multiple back end nodes.
- Typically the cluster will be configured with multiple redundant load-balancing front ends.
- all available servers process requests.
- Web servers, mail servers,..

HPC Clusters

- Low Latency Network
- Message passing libraries
- Parallel Filesystem
- HPC system software

Cluster Computing

High Performance Computing vs High Thruput Computing

High Performance (Capability)

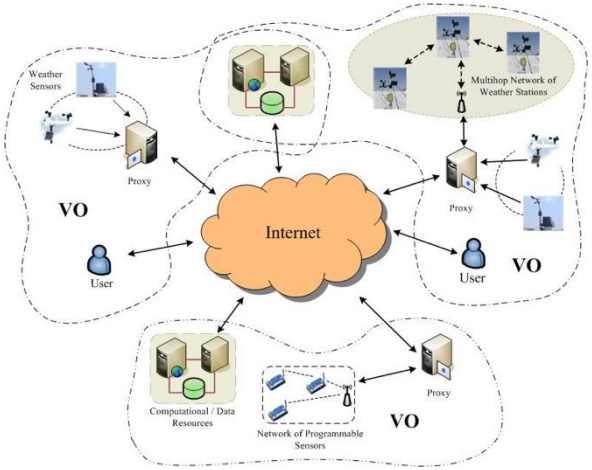
High Throughput (Capacity)

Fine-grained Applications
- Many-node
- Few concurrent runs
- High interconnect use

Course-grained Applications
- Single-node
- Many concurrent runs
- No interconnect use

Grid Computing (2000s)

Federating distributed resources



Characteristics

- Geographically distributed
- Multiple organizations sharing
- Heterogeneous resources
- WAN connections

Why It Didn't Become Cloud

- Complex middleware
- Unreliable resource availability
- No clear economic model
- Primarily scientific use

Cloud inherits: Resource virtualization, federation concepts

Utility Computing

Computing as a metered service

"Computing should be like electricity:
turn it on, use it, pay for what you consume"

The Economic Model

- Pay-per-use pricing (no upfront investment)
- On-demand provisioning (resources when needed)
- Service-level agreements (guaranteed performance)

Cloud inherits: The business model that makes it viable

The Cloud Synthesis (2006+)

Combining the best of all paradigms

Mainframes	→	Multi-tenancy, resource pooling
Clusters	→	Horizontal scaling, commodity HW
Grids	→	Virtualization, federation
Utility	→	Pay-per-use economics
Internet	→	Self-service APIs, global access

2006: AWS EC2 launches — cloud computing begins

Part 2

Distributed Systems Fundamentals

The principles that govern cloud behavior

What is a Distributed System?

> *"A collection of autonomous computing elements that appears to users as a single coherent system"*
>
> — Tanenbaum & Van Steen

Key insight:

- Multiple independent computers
- Working together
- Appearing as one system to the user

Every cloud service is a distributed system

Three Fundamental Properties

What makes distributed systems different

Concurrency

Components execute simultaneously

No Global Clock

Cannot perfectly synchronize time

Independent Failures

Parts fail while others continue

These properties make distributed systems fundamentally harder than single-machine systems

Applications of Distributed Computing

1. Some applications are **inherently distributed problems (**they are solved most easily using the means of distributed computing)
2. Computing intensive problems where **communications is limited or absent** (High Throughput Computing)
3. **Data Intensive problems:** computing task deal with a large amount or large size of data.
4. **Distributed computing allows for "scavenging"** By integrating the computers into a distributed system, the excess computing power can be made available to other users or applications (e.g. Condor)

Eight Fallacies of Distributed Computing

False assumptions developers make (Part 1 of 2)

1	**The network is reliable**	Networks fail, packets are lost
2	**Latency is zero**	Every call takes time (ms to seconds)
3	**Bandwidth is infinite**	Data transfer has limits and costs
4	**The network is secure**	Assume hostile environment

Eight Fallacies (continued)

Part 2 of 2

5	**Topology doesn't change**	Nodes come and go constantly
6	**There is one administrator**	Multiple teams, organizations
7	**Transport cost is zero**	Serialization, bandwidth cost money
8	**The network is homogeneous**	Different protocols, versions, vendors

Implications of the Fallacies

Design your cloud applications assuming ALL of these assumptions are FALSE

Practical implications:

- Implement retries with exponential backoff
- Use timeouts on all network calls
- Design for idempotency (safe to retry)
- Encrypt data in transit
- Use service discovery (not hardcoded addresses)

Three Properties of Distributed Systems

Eric Brewer (2000), Proven by Gilbert & Lynch (2002)

C

Consistency

Every read receives the most recent write (or an error)

A

Availability

Every request receives a response (no timeout/error)

P

Partition Tolerance

System operates despite network partitions

A network partition = nodes cannot communicate with each other

The CAP Theorem

During a network partition, a distributed system must choose between Consistency and Availability

CP: Choose Consistency

Refuse requests rather than return potentially stale data

AP: Choose Availability

Serve requests even if data might be stale

Continued in Part 2: CAP trade-offs, Consistency models, Consensus...